

- **Result:** In the non-private setting, modern machine learning models are typically pre-trained on a large amount of diverse data (of varying quality), and the weights are then fine-tuned on a smaller, higher-quality, task-specific dataset. This two-stage process lends itself very naturally to differential privacy: consider the pre-training dataset as “public” (indeed, these datasets are often obtained through public webscrapes, say, downloading Reddit comments, Wikipedia text, Project Gutenberg books, etc.) and pre-train as normal, but consider the smaller task-specific dataset to be sensitive (e.g., emails, medical documents, etc.) and thus do fine-tuning using differentially private stochastic gradient descent (DPSGD). We show that this approach can get surprisingly high utility for NLP tasks, at levels which are still useful, and in the same ballpark as non-private results. This is in comparison to other private ML tasks (e.g., image classification on the CIFAR-10 dataset), where the introduction of privacy (without any public data) can result in dramatic drops in utility, rendering the resulting models useless. Furthermore, we show that, empirically, larger models produce higher utility: both in terms of absolute utility, as well as smaller drops due to introducing privacy. This is in line with behaviour in the non-private case (where bigger models are also better), but contrary to the conventional wisdom in DP (where bigger models were thought to be worse, since more parameters results in more noise being introduced).
- **Significance:** One of the largest barriers to widespread adoption of differentially private ML is that the utility drop due to privacy is unacceptable. Our work demonstrates that large, public datasets can help overcome this hurdle, providing actually useful machine learning models which are trained privately with respect to the sensitive data. Indeed, our work popularized this paradigm and at this point, public pre-training and private fine-tuning is the de facto standard way of doing differentially private machine learning. This specific work directly led to deployments in practice at Microsoft, including the main LLM used in Microsoft Outlook’s email server.

Differentially Private Fine-tuning of Language Models*

Da Yu[†] Saurabh Naik[‡] Arturs Backurs[§] Sivakanth Gopi[§] Huseyin A. Inan[§]
 Gautam Kamath[¶] Janardhan Kulkarni[§] Yin Tat Lee^{||} Andre Manoel[§]
 Lukas Wutschitz[‡] Sergey Yekhanin[§] Huishuai Zhang^{**}

July 18, 2022

Abstract

We give simpler, sparser, and faster algorithms for differentially private fine-tuning of large-scale pre-trained language models, which achieve the state-of-the-art privacy versus utility tradeoffs on many standard NLP tasks. We propose a meta-framework for this problem, inspired by the recent success of highly parameter-efficient methods for fine-tuning. Our experiments show that differentially private adaptations of these approaches outperform previous private algorithms in three important dimensions: utility, privacy, and the computational and memory cost of private training. On many commonly studied datasets, the utility of private models approaches that of non-private models. For example, on the MNLI dataset we achieve an accuracy of 87.8% using RoBERTa-Large and 83.5% using RoBERTa-Base with a privacy budget of $\epsilon = 6.7$. In comparison, absent privacy constraints, RoBERTa-Large achieves an accuracy of 90.2%. Our findings are similar for natural language generation tasks. Privately fine-tuning with DART, GPT-2-Small, GPT-2-Medium, GPT-2-Large, and GPT-2-XL achieve BLEU scores of 38.5, 42.0, 43.1, and 43.8 respectively (privacy budget of $\epsilon = 6.8, \delta = 1e-5$) whereas the non-private baseline is 48.1. All our experiments suggest that larger models are better suited for private fine-tuning: while they are well known to achieve superior accuracy non-privately, we find that they also better maintain their accuracy when privacy is introduced.

1 Introduction

Deep learning models are well known to leak sensitive information about the dataset when trained using conventional methods (Shokri et al., 2017; Carlini et al., 2019, 2021). To combat this issue, models can instead be trained to guarantee differential privacy (DP) (Dwork et al., 2006b), a strong notion of data privacy which limits the influence of any individual training point on the final model. While DP is one of the few approaches capable of providing machine learning models with rigorous privacy guarantees, it generally comes at a cost in terms of test accuracy. One oft-cited explanation is that the constraint of DP necessitates much more training data (Tramèr and Boneh, 2021; Feldman, 2020; Brown et al., 2021). Unfortunately, more training data may be hard to acquire, particularly in settings where privacy is a concern.

Parallel to these developments, Transformer-based (Vaswani et al., 2017) large language models (LLMs), including the BERT (Devlin et al., 2019; Liu et al., 2019) and GPT (Radford et al., 2018, 2019; Brown et al., 2020) families, have enabled significant progress in natural language processing, achieving state-of-the-art accuracy in almost every task considered. These models are first pre-trained on an extremely large and diverse public dataset. The weights are then fine-tuned for each task of interest using a much smaller

*Aside from the first and second authors, all other authors are listed in alphabetical order.

[†]Sun Yat-sen University. yuda3@mail2.sysu.edu.cn. Work was done while an intern at Microsoft Research Asia.

[‡]Microsoft. {snaik, lukas.wutschitz}@microsoft.com.

[§]Microsoft Research. {arturs.backurs, sigopi, huseyin.inan, jakul, amonteioroman, yekhanin}@microsoft.com.

[¶]Cheriton School of Computer Science, University of Waterloo. g@csail.mit.edu. Supported by an NSERC Discovery Grant.

^{||}University of Washington and Microsoft Research. yintat@uw.edu.

^{**}Microsoft Research Asia. huzhang@microsoft.com.

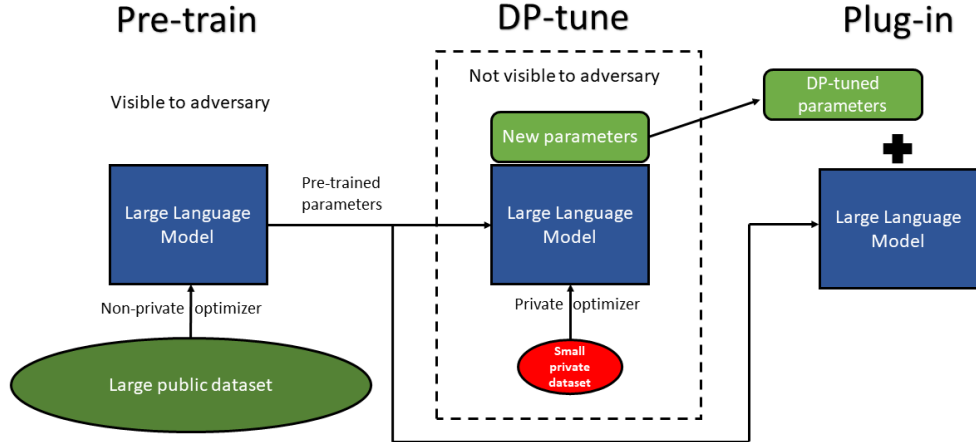


Figure 1: An illustration of our framework. First, the model is pre-trained on a large, public dataset. Next, new parameters are introduced and privately fine-tuned on a smaller, private, task-specific dataset. The original parameters are frozen during this process. Finally, the fine-tuned new parameters may be released publicly and plugged-in to the model for downstream tasks, while still preserving privacy of the private dataset.

Table 1: Accuracy of fine-tuning for downstream tasks with RoBERTa-Large (in %). Our results achieve accuracy comparable to full fine-tuning non-privately, while simultaneously guaranteeing differential privacy and modifying less than 1% of the parameters. We choose $\delta = 1e-5$ for SST-2 and QNLI and $\delta = 1e-6$ for MNLI and QQP due to their dataset sizes. Implementation details are in Section 4.1.

Method	MNLI	SST-2	QQP	QNLI	Avg.	Trained params
Non-private fine-tuning	90.2	96.4	92.2	94.7	93.4	100%
Our results ($\epsilon = 6.7$)	87.8	95.3	87.4	90.8	90.3	0.94%

task-specific dataset. For example, a single pre-trained GPT-family model may be fine-tuned for various downstream tasks, such as email reply suggestion, sentence completion in text editors, language translation, and more. This two-stage paradigm can naturally be adapted to solve tasks in private learning, automatically addressing the aforementioned data shortage issue via the massive scale of the public pre-training dataset. One may pre-train the model on public data as usual,¹ but then fine-tune the model *privately*.

Despite the success of these models, task-specific fine-tuning introduces a number of technical challenges. In the non-private setting, the immense size of LLMs makes it impractical to fine-tune the full model and store a separate copy of the parameters for hundreds of downstream tasks. Things only get worse with privacy, which leads to overheads in terms of running time, memory usage, and most importantly, accuracy. The magnitude of noise introduced to a model due to DP grows as the model size increases (Bassily et al., 2014; Abadi et al., 2016; Bun et al., 2014), which can overwhelm any signal for larger models. A recent line of work in the non-private literature has proposed parameter-efficient methods to alleviate the issues of storage and computational cost for fine-tuning (Houlsby et al., 2019; Li and Liang, 2021; Aghajanyan et al., 2020; Hu et al., 2021; Mahabadi et al., 2021). The main focus of our work is to explore parameter-efficiency in the context of private learning.

1.1 Our Contributions

Our primary contribution is to show that advanced parameter-efficient methods can lead to *simpler* and significantly improved algorithms for private fine-tuning. Our framework is illustrated in Figure 1. Our

¹Despite the fact that the pre-training data is public, there may nonetheless be privacy concerns related to personal or copyrighted data. However, since these pre-trained models have already been released, any associated privacy loss has already been incurred.

Table 2: Fine-tuning GPT-2 models on the DART dataset. We observe that larger models have better utility, both in absolute numbers, and in terms of preserving non-private utility. DP parameters are ($\epsilon = 6.8, \delta = 1e-5$).

Model	BLEU (DP)	BLEU (non-private)	Drop due to privacy
GPT-2-Medium	42.0	47.1	5.1
GPT-2-Large	43.1	47.5	4.4
GPT-2-XL	43.8	48.1	4.3

findings and contributions are summarized as follows:

- State-of-the-art utility and privacy.** Empirical evaluation of our algorithms reveals that they achieve state-of-the-art accuracy versus privacy tradeoffs, improving upon the previous best (Yu et al., 2021b). More importantly, for many fine-tuning tasks, the utility of models trained with DP approaches that of non-private models. For example, privately fine-tuning RoBERTa-Large on the MNLI data set (Williams et al., 2018), we achieve an accuracy of 87.8% with a privacy budget of ($\epsilon = 6.7, \delta = 1e-6$). Without privacy guarantees, RoBERTa-Large achieves an accuracy of 90.2% (GPT-3 is known to achieve 91.7% (Hu et al., 2021)); see Table 1 for a summary. We also explore private natural language generation tasks, fine-tuning GPT-2 models on the E2E dataset (Novikova et al., 2017). Again, the utility approaches non-private levels: we achieve a ROUGE-L score of 67.8 with GPT-2-Large and ($\epsilon = 6.0, \delta = 1e-5$), compared to 72.0 without privacy.
- Larger models are better.** Prior work has consistently shown that larger language models achieve better accuracy for downstream tasks. Our results give evidence that this phenomenon extends to the private setting. For example, on the MNLI dataset, RoBERTa-Base achieves an accuracy of 83.5% (versus 87.6% non-privately, a drop of 4.1%) whereas RoBERTa-Large achieves an accuracy of 87.8% (versus 90.2% non-privately, a drop of 2.4%), both under a privacy budget of ($\epsilon = 6.7, \delta = 1e-6$). Similarly, privately fine-tuning (using LoRA (Hu et al., 2021)) on DART ($\epsilon = 6.8, \delta = 1e-5$), GPT-2-Medium achieves a BLEU score of 42.0 (versus 47.1 non-privately, a drop of 5.1) while GPT-2-XL achieves a BLEU score of 43.8 (versus 48.1 non-privately, a drop of 4.3), see Table 2. Observe that utility improves with model size in two ways: both in terms of absolute numbers, as well as the drop incurred due to privacy. While the power of large models has been established in the non-private setting, we find this phenomenon quite surprising under DP. There is often a tension when choosing private model architectures: larger models may have higher capacity, but necessitate the introduction of more noise. Consequently, smaller and simpler private models achieve the better accuracy in several settings (Papernot et al., 2019; Tramèr and Boneh, 2021). In contrast, our experiments show that fine-tuning the biggest models achieves the best accuracy,² which we consider to be one of our main findings.
- Simpler, sparser, and faster.** Beyond accuracy concerns, DP requirements also lead to significant overheads in terms of computation and memory usage. The large number of parameters contributes to the high cost of training LLMs, and things get worse under privacy, which has been documented to increase training time by up to two orders of magnitude (Carlini et al., 2019; Subramani et al., 2021). The parameter-efficient approaches we employ partially offset this issue: as we only update a small fraction of the total number of parameters, training becomes considerably more computationally and memory efficient. Furthermore, as in the non-private setting, this framework leads to a modular design, where a single large pre-trained model can be augmented with lightweight modifications for each individual downstream task.

To the best of our knowledge, we are the first to fine-tune GPT-2-XL using differential privacy, the largest model (with 1.5B parameters) trained thus far using DP. Given our state-of-the-art results for a variety of standard NLP tasks using advanced fine-tuning techniques, we believe that our paper will serve

²An alternative perspective is that what we currently think of as “large” language models are relatively small, and we are yet to reach the point where the benefits of model size on accuracy are outweighed by the drawbacks.

as a benchmark for further work in this direction. For example, the best average accuracy achieved by the prior work of Yu et al. (2021b) on four standard NLP tasks in Table 1 is 83.9% using $\epsilon = 8$ (and the same δ as in Table 1), whereas we can achieve an average accuracy of 90.3% using $\epsilon = 6.7$ by a combination of better algorithms, larger models, and new privacy accounting techniques.

Finally, though recently considered elsewhere (see Section 5), we put further focus on the framing of public pre-training and private fine-tuning as an important conceptual direction in DP deep learning.

2 Preliminaries and Prior Algorithm Baselines

Recall the formal definition of differential privacy.

Definition 2.1 (Differential Privacy (DP) (Dwork et al., 2006b,a)). *A randomized algorithm $\mathcal{A}$ is (ϵ, δ) -differentially private if for any two neighboring datasets D and D' , which differ in exactly the data pertaining to a single user, and for all sets $\mathcal{S}$ of possible outputs: $\Pr[\mathcal{A}(D) \in \mathcal{S}] \leq e^\epsilon \Pr[\mathcal{A}(D') \in \mathcal{S}] + \delta$.*

We review prior techniques for private fine-tuning.

2.1 Full Fine-tuning via DPSGD

To train a machine learning model with privacy, the most popular algorithm is the celebrated DP stochastic gradient descent (DPSGD) (Song et al., 2013; Bassily et al., 2014; Abadi et al., 2016). This optimization method serves as a drop-in replacement for SGD, augmenting it with the addition of per-example gradient clipping and Gaussian noise addition steps. These two steps serve to limit and mask the contribution of a single example. Two key points to note are that a) per-example gradient clipping incurs significant computational and memory overheads in most implementations, and b) noise introduced due to privacy grows as the square-root as the number of model parameters. With this tool in place, the most basic fine-tuning strategy is to train all parameters using DPSGD.

2.2 Reparametrized Gradient Perturbation

To mitigate the limitations of DPSGD, a recent work of Yu et al. (2021b) introduced an elegant method called *reparametrized gradient perturbation* (RGP). RGP exploits the implicit low-rank structure in the gradient updates of SGD to substantially improve upon DPSGD. Specifically, they reparametrize each layer’s weight matrix W into $LR + \tilde{W}$, where L and R are low-rank gradient-carrier matrices and $\tilde{W}$ is the residual weight. The authors show that one can obtain a low-dimensional projection of W ’s gradient by taking gradients only of the low-rank matrices L and R (and not the high-rank $\tilde{W}$). Privacy is introduced by clipping and noising these low-dimensional gradients of L and R . While this low-dimensional projection loses some of the signal in W ’s gradient, it turns out to contain enough to still achieve high accuracy. At the same time, the low-dimensional gradients alleviate the aforementioned issues related to privatization, significantly reducing the memory consumption and noise introduced. Although RGP uses a low-rank update at each step, we empirically verify that its accumulated update is not of low stable rank and hence can not be compressed into small plug-in modules. Possible reasons include: 1) the low-rank subspaces of RGP are different at different updates; 2) the accumulated update of RGP contains all the added noises, which are of high stable rank.

3 Our Approach

3.1 A Meta-framework

We introduce our approach as a meta-framework for private deep learning, which abstracts the key principles of recent fine-tuning methods.

Suppose $f(W_{\text{PT}}; x)$ is a pre-trained model where W_{PT} are the pre-trained weights and x is any input. We create a new fine-tuning model

$$f_{\text{FT}}(W_{\text{PT}}, \theta; x) \tag{1}$$

which incorporates additional trainable parameters θ , where $\dim(\theta) \ll \dim(W_{\text{PT}})$. That is, the number of new parameters in θ is a small fraction of the original number of parameters in the pre-trained weights W_{PT} . Fine-tuning is done by running DPSGD on the additional parameters θ , while freezing the weights of pre-trained model W_{PT} . The new parameters are initialized to θ_0 such that

$$f_{\text{FT}}(W_{\text{PT}}, \theta_0; x) = f(W_{\text{PT}}; x). \quad (2)$$

The initialization condition (2) is very important, as it ensures that fine-tuning starts at the pre-trained model and improves it by modifying the parameters θ . Most fine-tuning methods are additive and have the following special form:

$$f_{\text{FT}}(W_{\text{PT}}, \theta; x) = f(W_{\text{PT}} + \pi(\theta); x), \quad (3)$$

i.e., they modify the pre-trained weights by adding a correction term $\pi(\theta)$ parametrized by θ .

Recent work in the non-private literature has described concrete instantiations of this framework (Houlsby et al., 2019; Mahabadi et al., 2021; Hu et al., 2021), which (crucially) are effective when $\dim(\theta) \ll \dim(W_{\text{PT}})$. In the non-private setting, such reparametrizations are useful for reducing the computation and memory required for fine-tuning, and enable lightweight and plug-in modifications to the base model for different downstream tasks. At the same time, they maintain (or sometimes surpass) the accuracy achieved by full fine-tuning.

We give some intuition as to why parameter-efficient methods could be more effective for private fine-tuning especially when private datasets are small. For simplicity, we assume that the fine-tuning method is additive as in (3), such that the fine-tuned weights $W_{\text{FT}} = W_{\text{PT}} + \pi(\theta)$. We can imagine that W_{FT} lies on a manifold passing through W_{PT} of very small dimension (equal to the dimension of θ) compared to the dimension of W_{PT} . Even if the parameters θ are very noisy due to the noise added during DPSGD, we will always stay in this manifold. In particular, we are not disturbing the pre-trained weights in most directions (those orthogonal to the manifold near W_{PT}). If we run DPSGD on all the weights instead, then we add noise in all directions, thus potentially unlearning the knowledge learned during pre-training, especially in low data regimes; see the discussion in Section 4.3 for more on this.

Besides substantial gains in the accuracy, the above method of reparametrization has several other advantages:

- A single pre-trained model such as BERT or GPT is generally applied to hundreds of downstream tasks via fine-tuning. Private fine-tuning using previous methods requires updating *all* parameters and storing a different copy of the fine-tuned model per task. This creates substantial overheads for storing and deploying, and can be very expensive in practice. On the other hand, the reparametrization (1) means that we only need to store a single pre-trained model that can be shared across many downstream tasks. Each downstream task requires only a small number of new parameters that can be plugged in.
- Differentially private training requires computing and storing per-example gradients, which increases the memory footprint. In our approach, however, learning is done in a much lower dimension, hence saving on the memory cost as compared to prior works.
- Finally, we expect that (1) also gives a more communication-efficient method of fine-tuning in distributed settings such as federated learning, due to the significantly smaller number of parameters learned during fine-tuning.

3.2 Instantiating the Meta-framework

In this section, we discuss a few ways to instantiate our meta-framework. This list is non-exhaustive, but covers the methods we employ in our experiments.

3.2.1 Fine-tuning via Low-Rank Adaptation

Low-Rank Adaptation (LoRA) (Hu et al., 2021) is an additive fine-tuning scheme as defined in (3). For each dense weight matrix W_{PT}^i of size $a \times b$ in the pre-trained network, we add a low-rank correction term $L^i R^i$, i.e.,

$$W^i = W_{\text{PT}}^i + L^i R^i, \quad (4)$$

where $L^i \in \mathbb{R}^{a \times r}$, $R^i \in \mathbb{R}^{r \times b}$ are new trainable parameters. [Hu et al. \(2021\)](#) apply this reparameterization only to the Transformer attention weights (W_q, W_v), and freeze all other weights (e.g., W_k and W_o and those in the feed-forward layers). The rank r is typically chosen to be small, e.g., $r = 4, 16, 64$. Since most parameters in Transformer architectures are dense weight matrices, choosing a small r results in a nearly square-root reduction in the number of parameters.

3.2.2 Fine-tuning via Adapters

[Houlsby et al. \(2019\)](#) propose adapter-based fine-tuning, in which we modify the architecture of the pre-trained model by adding new “adapter” layers after each attention and feed-forward layer. Adapter layers are bottleneck layers with residual connections. Specifically, given an input x , an adapter layer A performs

$$A(x) = U(\tau(D(x))) + x, \quad (5)$$

where U is an up-projection affine linear map, D is a down-projection affine linear map, and τ is a non-linear activation function such as the Gaussian error Linear Unit (GeLU) ([Hendrycks and Gimpel, 2016](#)). If x has dimension d , then $U \in \mathbb{R}^{d \times r}$, $D \in \mathbb{R}^{r \times d}$ for some $r \ll d$. Thus, the number of introduced parameters is significantly less than the number of parameters in the pre-trained model. When fine-tuning, the parameters of the original model are frozen, and only parameters of the adapter layers, as well as layer normalizations, are modified. Note that fine-tuning with adapters is not an additive fine-tuning framework as in (3), but is captured by the broader framework in (1).

3.2.3 Fine-tuning via Compacter

The recent work of [Mahabadi et al. \(2021\)](#) introduces Compacters (Compact adapters), a method which further improves the parameter efficiency of adapters. This is done by replacing the dense matrices in the up-projection U and down-projection D by tensor products of smaller matrices, thus reducing the number of trainable parameters. Specifically, they replace the dense matrix M_ℓ in the adapter layer ℓ by a low-rank parameterized hypercomplex multiplication (LPHM) layer, i.e., each dense matrix $M_\ell \in \mathbb{R}^{a \times b}$ is expressed as

$$M_\ell = \sum_{i=1}^n A_i \otimes (S_i^\ell T_i^\ell) \quad (6)$$

where $A_i \in \mathbb{R}^{n \times n}$, $S_i^\ell \in \mathbb{R}^{a/n \times k}$, $T_i^\ell \in \mathbb{R}^{k \times b/n}$ and $\otimes$ is the matrix Kronecker product. Note the matrices A_i are not indexed by the layer ℓ because these matrices are shared among all the adapter layers. Since each adapter layers has two dense matrices (one for up-projection and one for down-projection), if there are L adapter layers, this reduces the number of parameters from $L(2ab)$ to $L(2(a+b)k) + n^3$. In practice, a and b are chosen to be either the model dimension d or the intermediate representation dimension r in the adapters, n is typically chosen to be a small constant such as $n = 2, 4, 8, 12$ and k is chosen to be 1.

3.2.4 Why Does Parameter-Efficient Tuning Work?

Theoretical explanation of the success of parameter-efficient fine-tuning methods is an active area of research in deep learning. Indeed, since trends have consistently shown that model accuracy increases with size, how can one achieve competitive accuracy while fine-tuning less than 1% of the parameters? One popular hypothesis is *intrinsic dimensionality* ([Li et al., 2018](#)), which posits that the minimum number of parameters needed to train a machine learning model may be much less than the total number of model parameters. [Aghajanyan et al. \(2020\)](#) explore this hypothesis in the context of fine-tuning LLMs, showing that one can achieve most of their accuracy by training only a very small number of parameters (chosen via a random projection). Perhaps surprisingly, they find that as *the model size increases, intrinsic dimension decreases*, in the limit exhibiting zero-shot learning. While we did not explore this hypothesis in the context of DP due to computational restrictions, we believe it may be an interesting lens through which one can understand the effectiveness of private parameter-efficient fine-tuning.

Table 3: Memory and speed comparison for RoBERTa-Large. The rank is chosen as $r = 16$ for RGP and LoRA. The speed is measured by the wall-clock time for training one epoch of the SST-2 dataset on a single Tesla V100 GPU with gradient accumulation for batch size 2000.

Method	Memory (GB)	Speed (seconds per epoch)
Full fine-tuning (DPSGD)	27.9	715
RGP	9.1	296
DP LoRA	6.1	271

3.3 Comparison with Baseline Algorithms

We highlight some key algorithmic differences between our proposed methods and the baselines of full fine-tuning and RGP.

- DPSGD and RGP both require updating all parameters of the pre-trained model, whereas our proposed methods update only a tiny fraction (between 0.05% and 1%). The rightmost columns of Tables 4 and 5 list the number of parameters trained by these algorithms.
- RGP performs a low-rank decomposition of weight matrices which is very similar to LoRA, though there are subtle differences. Recall that in RGP, at the beginning of each iteration t , the historical weight matrix W_{t-1} is decomposed to find a low-rank product LR . The gradients computed on L and R are then projected back to the full parameter space to perform the descent step. Hence, RGP does not keep the pre-trained weights frozen during the learning process.

LoRA can be viewed as a simplification of RGP. LoRA reparametrizes $W_{FT} := W_{PT} + LR$, where the pre-trained weight matrix W_{PT} is frozen during training. Hence, compared to RGP, LoRA eliminates the decomposition and the projection to the full parameter space at each iteration, simplifying the implementation and reducing the running time and memory cost. This is summarized in Table 3. We observe that DP LoRA reduces the memory cost by about 33% and the training speed by 8%. As we will see, this simplification also results in improved utility.

- Neither full fine-tuning nor RGP fall into our meta-framework described by (1). Thus, if a pre-trained model is to be applied to several downstream tasks, one must store a separate set of weights for each task, incurring a significant memory cost and losing the plug-in functionality. In contrast, our methods are much more lightweight.

4 Experiments

We experimentally evaluate our methods for DP fine-tuning to demonstrate their utility, privacy, and parameter-efficiency. We investigate both language understanding and text generation tasks, using RoBERTa and GPT-2 models, to establish that our techniques are applicable to a variety of tasks and model architectures.³

4.1 Fine-Tuning for Language Understanding Tasks

We first compare our methods with state-of-the-art fine-tuning algorithms using models from the BERT family, which was used in the prior work (Yu et al., 2021b). Specifically, we use RoBERTa models (Liu et al., 2019),⁴ which are pre-trained on public data collected from the web. RoBERTa-Base has 125M parameters and RoBERTa-Large has 355M parameters. We fine-tune the pre-trained models on four tasks: MNLI, QQP, QNLI and SST-2 from the GLUE benchmark (Wang et al., 2018), following Yu et al. (2021b).

Implementation Details: For fine-tuning with adapters, we may choose the intermediate representation dimension r , shared across all adapter layers. Similarly, for fine-tuning with Compacter, we can choose both

³Code for our experiments is available at <https://github.com/huseyinatahaninan/Differentially-Private-Fine-tuning-of-Language-Models>.

⁴The model can be found at <https://github.com/pytorch/fairseq/tree/master/examples/roberta>.

Table 4: Accuracy for fine-tuning downstream tasks with RoBERTa-Base (in %). The privacy parameters are $\epsilon = 6.7$, and $\delta = 1\text{e-}5$ for SST-2 and QNLI and $1\text{e-}6$ for MNLI and QQP. Bold indicates the best accuracy with DP. Numbers for non-private fine-tuning are from Liu et al. (2019).

Method		MNLI	SST-2	QQP	QNLI	Avg.	Trained params
Full	w/o DP	87.6	94.8	91.9	92.8	91.8	100%
	DP	53.1	82.6	74.4	63.9	68.5	
LoRA	w/o DP	87.5	95.1	90.8	93.3	91.7	0.24%
RGP ⁵	DP	80.1	91.6	85.5	87.2	86.1	100%
Adapter	DP	83.4	92.5	85.6	87.5	87.3	1.4% ($r = 48$)
Compacter	DP	82.6	92.3	84.7	85.1	86.2	0.055% ($r = 96, n = 8$)
LoRA	DP	83.5	92.2	85.7	87.3	87.2	0.94% ($r = 16$)

Table 5: Accuracy for fine-tuning downstream tasks with RoBERTa-Large (in %). The privacy parameters are $\epsilon = 6.7$, and $\delta = 1\text{e-}5$ for SST-2 and QNLI and $\delta = 1\text{e-}6$ for MNLI and QQP. Bold indicates the best accuracy with DP. Numbers for non-private fine-tuning are from Liu et al. (2019).

Method		MNLI	SST-2	QQP	QNLI	Avg.	Trained params
Full	w/o DP	90.2	96.4	92.2	94.7	93.4	100%
LoRA	w/o DP	90.6	96.2	91.6	94.9	93.3	0.23%
RGP	DP	86.1	93.0	86.7	90.0	88.9	100%
Adapter	DP	87.7	93.9	86.3	90.7	89.7	1.4% ($r = 48$)
Compacter	DP	87.5	94.2	86.2	90.2	89.5	0.053% ($r = 96, n = 8$)
LoRA	DP	87.8	95.3	87.4	90.8	90.3	0.94% ($r = 16$)

the intermediate representation dimension r and the Kronecker product kernel dimension n in (6). For LoRA fine-tuning, we add bottleneck branches for both the attention layers and the feedforward layers, which differs slightly from the addition of bottleneck branches for only the W_q and W_v matrices of the attention layers as done by Hu et al. (2021). Given the same bottleneck representation dimension r in (4), our new implementation uses twice as many trainable parameters as the original paper, and achieves some improvements for learning with DP. We perform privacy accounting using the PRV Accountant from Gopi et al. (2021), which currently provides the tightest bounds.

Hyperparameter choice: Given the large number of hyperparameter choices, e.g., the intermediate representation dimension, learning rate, weight decay, privacy delta, and model size, an exhaustive grid search over all hyperparameters is expensive, due to the model sizes. Our hyperparameter choices are informed by prior work and are as follows. For privacy parameters, we use $\delta = 1\text{e-}5$ for SST-2 and QNLI and $\delta = 1\text{e-}6$ for QQP and MNLI due to their dataset sizes, and use noise multipliers 0.92, 0.83, 0.66 and 0.65 for SST-2, QNLI, QQP and MNLI, respectively, which is the same as Yu et al. (2021b)⁶. The clipping threshold of per-example gradients is 10 for all methods. For adapters and Compacter, we follow suggestions in the original papers and choose r from a set $\{16, 48, 96\}$ and n from a set $\{4, 8, 12\}$. For LoRA, we choose the best-performing rank r from the set $\{4, 16, 48, 64\}$. The best performing hyperparameters are noted in Tables 4 and 5. We use batch size 2000 and train with half-precision for 20 epochs. We use the optimizer AdamW (Loshchilov and Hutter, 2019) with weight decay $1\text{e-}2$ and search over four learning rates $\{5\text{e-}4, 1\text{e-}3, 2\text{e-}3, 5\text{e-}3\}$. In Appendix B, we show the proposed algorithms perform well for a wide range of hyperparameters.

Results: We report the prediction accuracy on four tasks in Tables 4 and 5. Our experiments using RoBERTa-Base serve as a direct comparison to the work of Yu et al. (2021b) who only trained the base model, whereas RoBERTa-Large experiments demonstrate the significance of using larger models. We could not report the numbers for full fine-tuning using DPSGD on RoBERTa-Large due to running time and memory costs; see the discussion at the end of this section. We summarize our key findings:

⁵We report RoBERTa-Base numbers from <https://github.com/dayu11/Differentially-Private-Deep-Learning>, by the authors of Yu et al. (2021b). Though the paper itself only reports results on BERT-Base, we cite their paper to also reference the RoBERTa numbers.

⁶In Appendix A, we evaluate the proposed framework with various choices of privacy parameters.

- On *all* datasets, our methods achieve the best accuracy while training a only tiny fraction of parameters; larger models give significant improvements.
- Noticeable improvements in the privacy parameter ε versus Yu et al. (2021b) are primarily due to new privacy accountants based on Fourier-based numerical composition (Koskela et al., 2020, 2021; Gopi et al., 2021); we use the PRV Accountant from Gopi et al. (2021) since it is the most efficient.
- Private adapters provide the best average performance for RoBERTa-Base, whereas LoRA outperforms all other methods for RoBERTa-Large.

Remark 4.1. A concurrent work by Li et al (Li et al., 2022) show that using a larger batch size and training with full-precision improves the performance of DPSGD, and obtains similar performance as our algorithm. Thus, poor performance of DPSGD in our experiments is due to the suboptimal choice of hyperparameters and also due to precision issues, although we use same hyperparameters for all the algorithms. We run new experiments with hyperparameters of (Li et al., 2022) in full precision mode, and get improvements around 1% even for our algorithms. We report these findings in Appendix C.

4.2 Fine-tuning for Natural Language Generation (NLG)

Next, we study private fine-tuning for text generation problems using the GPT-2 series of models on the End-2-End (E2E) NLG challenge (Novikova et al., 2017) and DART (Nan et al., 2021), two primary benchmarks used in recent works on non-private fine-tuning (Hu et al., 2021; Li and Liang, 2021). We use GPT-2-Small (117M parameters), GPT-2-Medium (345M parameters), GPT-2-Large (774M parameters), and GPT-2-XL (1.5B parameters).⁷ To the best of our knowledge, we are the first to privately fine-tune for E2E-DART or fine-tune GPT-2-XL. The purpose of this section is not to evaluate various fine-tuning algorithms, but to show that private fine-tuning is competitive with non-private fine-tuning for text generation problems. Due to the high cost of training, we report experimental results only for fine-tuning (private and non-private) with LoRA. We think that all fine-tuning methods in this paper should achieve comparable accuracy.

E2E NLG challenge: The E2E dataset was introduced by Novikova et al. (2017), and contains template-like information in the restaurant domain to be mapped to natural language with end-to-end training. The dataset consists of 42K training samples, 4.6K validation samples, and 4.6K test samples.

DART: DART was introduced as an open-domain data-to-text dataset by Nan et al. (2021). The dataset consists of 62K training samples, 6.9K validation samples, and 12K test samples. In comparison to E2E, the dataset is larger and the task is more challenging.

We use standard metrics such as BLEU, ROUGE-L, etc., used in (Hu et al., 2021) for measuring the quality of predictions.

Hyperparameter choice: For LoRA, we choose the bottleneck rank $r = 4$ in (4) and fine-tune W_q and W_v matrices of the attention layers as in the original paper. We optimize using AdamW with learning rate $4e-4$, weight decay $1e-2$ and train our models for 20 epochs. We use batch size 128 for the experiments on E2E and batch size 256 for the experiments on DART. We take the gradient clipping parameter to be 1.0 and the noise multiplier to be 0.6 for the accountant in Gopi et al. (2021), achieving $\varepsilon = 6.0, \delta = 1e-5$ on E2E and $\varepsilon = 6.8, \delta = 1e-5$ on DART.

Results: The results of our experiments are summarized in the Table 6 and 7, which reiterate the main themes of our work: private fine-tuning with parameter-efficient approaches perform close to their non-private counterparts and show consistent improvement in the utility as model size increases. Note that on E2E dataset, although the validation perplexity improves as the model becomes larger, the metrics seem to saturate going from large to XL for both private and non-private cases. On the other hand, for DART dataset both validation perplexity and the metric improve as the model size increases.

4.3 How Bad is DPSGD?

While our experiments indicate that full fine-tuning does not achieve competitive performance, there could be a choice of hyperparameters that improves upon the reported numbers, e.g., “mega” batch sizes (in the millions) in Anil et al. (2021); Li et al. (2022). We note that our main message is that one does not need

⁷https://huggingface.co/transformers/model_doc/gpt2.html.

Table 6: Metrics on the E2E NLG task. Non-DP results from Hu et al. (2021), except for GPT-2-XL, which was not reported in the paper. We ran GPT-2-XL with hyperparameters presented in Hu et al. (2021). Bold indicates the best accuracy with DP. DP parameters are ($\varepsilon = 6.0, \delta = 1e-5$). Val perp stands for validation perplexity.

Method	Val perp	BLEU	NIST	MET	ROUGE-L	CIDEr
GPT-2-Small + DP	4.51	63.8	7.19	39.5	67.5	1.87
GPT-2-Medium + DP	4.02	65.5	8.45	42.7	67.9	2.23
GPT-2-Large + DP	3.87	66.7	8.63	44.0	67.8	2.33
GPT-2-XL + DP	3.79	66.1	8.53	43.0	68.1	2.28
GPT-2-Medium	3.19	70.4	8.85	46.8	71.8	2.53
GPT-2-Large	3.06	70.4	8.89	46.8	72.0	2.47
GPT-2-XL	3.01	69.4	8.78	46.2	71.5	2.49

Table 7: Metrics on the DART dataset. Non-DP results from Hu et al. (2021), except for GPT-2-XL, which was not reported in the paper. We ran GPT-2-XL with hyperparameters presented in Hu et al. (2021). Bold indicates the best accuracy with DP. DP parameters are ($\varepsilon = 6.8, \delta = 1e-5$). Val perp stands for validation perplexity. Unlike all other metrics, the lower the TER metric is the better for the performance of the model.

Method	Val perp	BLEU	MET	TER
GPT-2-Small + DP	3.82	38.5	0.34	0.53
GPT-2-Medium + DP	3.30	42.0	0.36	0.51
GPT-2-Large + DP	3.10	43.1	0.36	0.5
GPT-2-XL + DP	3.00	43.8	0.37	0.5
GPT-2-Medium	2.67	47.1	0.39	0.46
GPT-2-Large	2.89	47.5	0.39	0.45
GPT-2-XL	2.83	48.1	0.39	0.46

to fine-tune all parameters to achieve the best accuracy. Nevertheless, it is interesting to wonder if full fine-tuning with DPSGD can match the accuracy of parameter-efficient methods. A positive answer would imply that private and non-private fine-tuning conceptually mirror each other.

Update: A concurrent work by Li et al (Li et al., 2022) answer our question and show that full fine-tuning via DPSGD also achieves same performance as the parameter efficient methods used in this paper. These two works raise an intriguing theoretical question: Why does magnitude of the noise added by DPSGD has no impact on the accuracy vs privacy tradeoffs when fine-tuning? Explaining this phenomenon would deepen our understanding of private learning.

5 Related Work

More on DP learning: Some work studies private language models on more traditional architectures such as LSTMs (Hochreiter and Schmidhuber, 1997), either training with DPSGD (McMahan et al., 2018; Carlini et al., 2019) or related heuristics (Ramaswamy et al., 2020). Though pre-training on public data is suggested (McMahan et al., 2018), public data appears to only be used in one of these works for honest hyperparameter selection (Ramaswamy et al., 2020). A few more recent works consider training LLMs with DP. Anil et al. (2021) privately train BERT-Large from scratch, compared to our work which focuses on private fine-tuning. (Hoory et al., 2021; Basu et al., 2021) perform private full fine-tuning of BERT models. Hoory et al. (2021) achieve accuracy which is comparable to the non-private model, but additionally supplement the public pre-training data with additional domain-relevant material, while we use off-the-shelf pre-trained models. Basu et al. (2021) observe significant drops in utility, compared to our parameter-efficient methods which do not. While Kerrigan et al. (2020) consider public pre-training and private fine-tuning, their experiments are on much smaller architectures (i.e., feedforward networks with three hidden layers). A simultaneous work of Ginart et al. (2022) investigates private *prediction* (rather than learning) for next-token prediction. A subsequent work by Senge et al. (2021) also investigates the effect of private fine-tuning on

various NLP tasks.

Our investigation fits more broadly into a line of work employing public data for private data analysis. Some works on image classification consider pre-training on a large public dataset and fine-tuning on a smaller private dataset (Abadi et al., 2016; Papernot et al., 2019; Tramèr and Boneh, 2021; Luo et al., 2021). In particular, Luo et al. (2021) investigate the role of parameter efficiency in private fine-tuning ResNet models, and propose strategies to choose which parameters to fine-tune. One line of work uses unlabeled public data to train a student model (Papernot et al., 2017, 2018; Bassily et al., 2018), including one work simultaneous to our own for natural language generation Tian et al. (2022). Another recent idea uses a small amount of public data to identify a lower-dimensional subspace of the gradients in which to perform private descent (Zhou et al., 2021; Yu et al., 2021a; Kairouz et al., 2021). A simultaneous work of Amid et al. (2021) uses public data in the mirror map for a private mirror descent algorithm. Finally, other works (both theoretical and experimental) investigate the role of public data in private query release, synthetic data generation, and prediction (Ji and Elkan, 2013; Beigel et al., 2016; Alon et al., 2019; Nandi and Bassily, 2020; Bassily et al., 2020a,b; Liu et al., 2021).

In a concurrent work, Li et al. (2022) also investigate DP fine-tuning of LLMs. In several cases, their results demonstrate qualitatively similar findings as ours. While our experiments focus primarily on parameter-efficient fine-tuning methods, interestingly, they show that private full fine-tuning can also achieve comparable utility if the experimental setup is configured properly, e.g., using suitable hyperparameters. In Appendix C, we run experiments under the setup in Li et al. (2022). We show their setup can also improve the performance of our methods.

More on Fine-tuning: There exist other parameter-efficient tuning methods which we did not evaluate in our work. Some of these include random subspace projection (exploiting intrinsic dimensionality (Li et al., 2018; Aghajanyan et al., 2020)), prefix and prompt tuning (Li and Liang, 2021; Lester et al., 2021), tuning only biases (Cai et al., 2020; Ben Zaken et al., 2021), and other architecture variants including Adapters (Pfeiffer et al., 2021; Rücklé et al., 2020). Other works investigate lightweight methods for adapting language models to different tasks (e.g., Dathathri et al. (2020)). An interesting direction for future work is to see whether parameter-efficient tuning approaches specifically designed for the private setting can achieve higher utility. We also mention zero-shot learning, in which no task-specific dataset is required and thus perfect privacy is achieved. Currently, zero-shot approaches achieve low utility compared to fine-tuning, though it is possible that future models may narrow this gap.

6 Conclusion

So far, DP deep learning has focused on training models from scratch. The spectacular success of transfer learning in real-world applications, however, shows that private fine-tuning is an equally pertinent problem to study and deserves more attention. We show that by combining recent advances in NLP, parameter-efficiency, privacy accounting, and using larger models, one can privately fine-tune models whose utility approaches that of non-private models. We hope our work inspires more study on the core problem of private fine-tuning, which we believe to be a central direction for research in private machine learning, leading to more interaction between the LLM and DP communities.

Acknowledgments

The authors would like to thank Rabeeh Karimi Mahabadi for sharing hyperparameters based on experiments in Mahabadi et al. (2021) and Xuechen Li for sharing the experimental setup and many suggestions about mixed-precision training. Janardhan Kulkarni would like to thank Edward Hu for sharing ideas on fine-tuning.

References

- Martin Abadi, Andy Chu, Ian Goodfellow, H Brendan McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang. Deep learning with differential privacy. In *Proceedings of the 2016 ACM Conference on Computer and Communications Security, CCS '16*, pages 308–318, New York, NY, USA, 2016. ACM.
- Armen Aghajanyan, Luke Zettlemoyer, and Sonal Gupta. Intrinsic dimensionality explains the effectiveness of language model fine-tuning. *arXiv preprint arXiv:2012.13255*, 2020.
- Noga Alon, Raef Bassily, and Shay Moran. Limits of private learning with access to public data. In *Advances in Neural Information Processing Systems 32*, NeurIPS '19, pages 10342–10352. Curran Associates, Inc., 2019.
- Ehsan Amid, Arun Ganesh, Rajiv Mathews, Swaroop Ramaswamy, Shuang Song, Thomas Steinke, Vinith M Suriyakumar, Om Thakkar, and Abhradeep Thakurta. Public data-assisted mirror descent for private model training. *arXiv preprint arXiv:2112.00193*, 2021.
- Rohan Anil, Badih Ghazi, Vineet Gupta, Ravi Kumar, and Pasin Manurangsi. Large-scale differentially private BERT. *arXiv preprint arXiv:2108.01624*, 2021.
- Raef Bassily, Adam Smith, and Abhradeep Thakurta. Private empirical risk minimization: Efficient algorithms and tight error bounds. In *Proceedings of the 55th Annual IEEE Symposium on Foundations of Computer Science, FOCS '14*, pages 464–473, Washington, DC, USA, 2014. IEEE Computer Society.
- Raef Bassily, Om Thakkar, and Abhradeep Guha Thakurta. Model-agnostic private learning. In *Advances in Neural Information Processing Systems 31*, NeurIPS '18, pages 7102–7112. Curran Associates, Inc., 2018.
- Raef Bassily, Albert Cheu, Shay Moran, Aleksandar Nikolov, Jonathan Ullman, and Steven Wu. Private query release assisted by public data. In *Proceedings of the 37th International Conference on Machine Learning, ICML '20*, pages 695–703. JMLR, Inc., 2020a.
- Raef Bassily, Shay Moran, and Anupama Nandi. Learning from mixtures of private and public populations. In *Advances in Neural Information Processing Systems 33*, NeurIPS '20, pages 2947–2957. Curran Associates, Inc., 2020b.
- Priyam Basu, Tiasa Singha Roy, Rakshit Naidu, Zumrut Muftuoglu, Sahib Singh, and Fatemehsadat Mireshghallah. Benchmarking differential privacy and federated learning for BERT models. *arXiv preprint arXiv:2106.13973*, 2021.
- Amos Beimel, Kobbi Nissim, and Uri Stemmer. Private learning and sanitization: Pure vs. approximate differential privacy. *Theory of Computing*, 12(1):1–61, 2016.
- Elad Ben Zaken, Shauli Ravfogel, and Yoav Goldberg. Bitfit: Simple parameter-efficient fine-tuning for transformer-based masked language-models. *arXiv preprint arXiv:*, 2021.
- Gavin Brown, Mark Bun, Vitaly Feldman, Adam Smith, and Kunal Talwar. When is memorization of irrelevant training data necessary for high-accuracy learning? In *Proceedings of the 53rd Annual ACM Symposium on the Theory of Computing, STOC '21*, pages 123–132, New York, NY, USA, 2021. ACM.
- Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems 33*, NeurIPS '20. Curran Associates, Inc., 2020.
- Mark Bun, Jonathan Ullman, and Salil Vadhan. Fingerprinting codes and the price of approximate differential privacy. In *Proceedings of the 46th Annual ACM Symposium on the Theory of Computing, STOC '14*, pages 1–10, New York, NY, USA, 2014. ACM.

- Han Cai, Chuang Gan, Ligeng Zhu, and Song Han. TinyTL: Reduce memory, not parameters for efficient on-device learning. In *Advances in Neural Information Processing Systems 33*, NeurIPS '20, pages 11285–11297. Curran Associates, Inc., 2020.
- Nicholas Carlini, Chang Liu, Úlfar Erlingsson, Jernej Kos, and Dawn Song. The secret sharer: Evaluating and testing unintended memorization in neural networks. In *28th USENIX Security Symposium*, USENIX Security '19, pages 267–284. USENIX Association, 2019.
- Nicholas Carlini, Florian Tramèr, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom Brown, Dawn Song, Úlfar Erlingsson, Alina Oprea, and Colin Raffel. Extracting training data from large language models. In *30th USENIX Security Symposium*, USENIX Security '21, pages 2633–2650. USENIX Association, 2021.
- Sumanth Dathathri, Andrea Madotto, Janice Lan, Jane Hung, Eric Frank, Piero Molino, Jason Yosinski, and Rosanne Liu. Plug and play language models: A simple approach to controlled text generation. In *Proceedings of the 8th International Conference on Learning Representations*, ICLR '20, 2020.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, NAACL-HLT '19, pages 4171–4186, 2019.
- Cynthia Dwork, Krishnaram Kenthapadi, Frank McSherry, Ilya Mironov, and Moni Naor. Our data, ourselves: Privacy via distributed noise generation. In *Proceedings of the 24th Annual International Conference on the Theory and Applications of Cryptographic Techniques*, EUROCRYPT '06, pages 486–503, Berlin, Heidelberg, 2006a. Springer.
- Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. Calibrating noise to sensitivity in private data analysis. In *Proceedings of the 3rd Conference on Theory of Cryptography*, TCC '06, pages 265–284, Berlin, Heidelberg, 2006b. Springer.
- Vitaly Feldman. Does learning require memorization? a short tale about a long tail. In *Proceedings of the 52nd Annual ACM Symposium on the Theory of Computing*, STOC '20, pages 954–959, New York, NY, USA, 2020. ACM.
- Antonio Ginart, Laurens van der Maaten, James Zou, and Chuan Guo. Submix: Practical private prediction for large-scale language models. *arXiv preprint arXiv:2201.00971*, 2022.
- Sivakanth Gopi, Yin Tat Lee, and Lukas Wutschitz. Numerical composition of differential privacy. *arXiv preprint arXiv:2106.02848*, 2021.
- Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*, 2016.
- Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- Shlomo Hoory, Amir Feder, Avichai Tendler, Alon Cohen, Sofia Erell, Itay Laish, Hootan Nakhost, Uri Stemmer, Ayelet Benjamini, Avinatan Hassidim, and Yossi Matias. Learning and evaluating a differentially private pre-trained language model. In *Proceedings of the Third Workshop on Privacy in Natural Language Processing*, PrivateNLP '21, pages 21–29, 2021.
- Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for nlp. In *International Conference on Machine Learning*, pages 2790–2799. PMLR, 2019.
- Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

- Zhanglong Ji and Charles Elkan. Differential privacy based on importance weighting. *Machine Learning*, 93(1):163–183, 2013.
- Peter Kairouz, Mónica Ribero, Keith Rush, and Abhradeep Thakurta. (nearly) dimension independent private ERM with adagrad rates via publicly estimated subspaces. In *Proceedings of the 34th Annual Conference on Learning Theory, COLT '21*, pages 2717–2746, 2021.
- Gavin Kerrigan, Dylan Slack, and Jens Tuyls. Differentially private language models benefit from public pre-training. *arXiv preprint arXiv:2009.05886*, 2020.
- Antti Koskela, Joonas Jälkö, and Antti Honkela. Computing tight differential privacy guarantees using fft. In *International Conference on Artificial Intelligence and Statistics*, pages 2560–2569. PMLR, 2020.
- Antti Koskela, Joonas Jälkö, Lukas Prediger, and Antti Honkela. Tight differential privacy for discrete-valued mechanisms and for the subsampled gaussian mechanism using fft. In *International Conference on Artificial Intelligence and Statistics*, pages 3358–3366. PMLR, 2021.
- Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, EMNLP '21. Association for Computational Linguistics, 2021.
- Chunyuan Li, Heerad Farkhoor, Rosanne Liu, and Jason Yosinski. Measuring the intrinsic dimension of objective landscapes. In *Proceedings of the 6th International Conference on Learning Representations, ICLR '18*, 2018.
- Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, ACL-IJCNLP '21, pages 4582–4597, 2021.
- Xuechen Li, Florian Tramèr, Percy Liang, and Tatsunori Hashimoto. Large language models can be strong differentially private learners. In *Proceedings of the 10th International Conference on Learning Representations, ICLR '22*, 2022.
- Terrance Liu, Giuseppe Vietri, Thomas Steinke, Jonathan Ullman, and Steven Wu. Leveraging public data for practical private query release. In *Proceedings of the 38th International Conference on Machine Learning, ICML '21*, pages 6968–6977. JMLR, Inc., 2021.
- Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. RoBERTa: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *Proceedings of the 7th International Conference on Learning Representations, ICLR '19*, 2019.
- Zelun Luo, Daniel J Wu, Ehsan Adeli, and Li Fei-Fei. Scalable differential privacy with sparse network finetuning. In *Proceedings of the 2021 IEEE Computer Society Conference on Computer Vision and Pattern Recognition, CVPR '21*, pages 5059–5068, Washington, DC, USA, 2021. IEEE Computer Society.
- Rabeeh Karimi Mahabadi, James Henderson, and Sebastian Ruder. Compacter: Efficient low-rank hyper-complex adapter layers. *arXiv preprint arXiv:2106.04647*, 2021.
- H Brendan McMahan, Daniel Ramage, Kunal Talwar, and Li Zhang. Learning differentially private recurrent language models. In *Proceedings of the 6th International Conference on Learning Representations, ICLR '18*, 2018.
- Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, et al. Mixed precision training. *arXiv preprint arXiv:1710.03740*, 2017.

- Linyong Nan, Dragomir Radev, Rui Zhang, Amrit Rau, Abhinand Sivaprasad, Chiachun Hsieh, Xiangru Tang, Aadit Vyas, Neha Verma, Pranav Krishna, Yangxiaokang Liu, Nadia Irwanto, Jessica Pan, Faiaz Rahman, Ahmad Zaidi, Mutethia Mutuma, Yasin Tarabar, Ankit Gupta, Tao Yu, Yi Chern Tan, Xi Victoria Lin, Caiming Xiong, Richard Socher, and Nazneen Fatema Rajani. DART: open-domain structured data record to text generation. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT '21*, pages 432–447. Association for Computational Linguistics, 2021.
- Anupama Nandi and Raef Bassily. Privately answering classification queries in the agnostic PAC model. In *Proceedings of the 31st International Conference on Algorithmic Learning Theory, ALT '20*, pages 687–703. JMLR, Inc., 2020.
- Jekaterina Novikova, Ondřej Dušek, and Verena Rieser. The E2E dataset: New challenges for end-to-end generation. In *Proceedings of the 18th Annual SIGdial Meeting on Discourse and Dialogue, SIGDIAL '17*, pages 201–206. Association for Computational Linguistics, 2017.
- Myle Ott, Sergey Edunov, David Grangier, and Michael Auli. Scaling neural machine translation. In *Proceedings of the Third Conference on Machine Translation (WMT)*, 2018.
- Nicolas Papernot, Martín Abadi, Ulfar Erlingsson, Ian Goodfellow, and Kunal Talwar. Semi-supervised knowledge transfer for deep learning from private training data. In *Proceedings of the 5th International Conference on Learning Representations, ICLR '17*, 2017.
- Nicolas Papernot, Shuang Song, Ilya Mironov, Ananth Raghunathan, Kunal Talwar, and Úlfar Erlingsson. Scalable private learning with PATE. In *Proceedings of the 6th International Conference on Learning Representations, ICLR '18*, 2018.
- Nicolas Papernot, Steve Chien, Shuang Song, Abhradeep Thakurta, and Ulfar Erlingsson. Making the shoe fit: Architectures, initializations, and tuning for learning with privacy. <https://openreview.net/forum?id=rJg851rYwH>, 2019.
- Jonas Pfeiffer, Aishwarya Kamath, Andreas Rücklé, Kyunghyun Cho, and Iryna Gurevych. Adapterfusion: Non-destructive task composition for transfer learning. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume, EACL '21*, pages 487–503. Association for Computational Linguistics, 2021.
- Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training, 2018.
- Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners, 2019.
- Swaroop Ramaswamy, Om Thakkar, Rajiv Mathews, Galen Andrew, H Brendan McMahan, and Françoise Beaufays. Training production language models without memorizing user data. *arXiv preprint arXiv:2009.10031*, 2020.
- Andreas Rücklé, Gregor Geigle, Max Glockner, Tilman Beck, Jonas Pfeiffer, Nils Reimers, and Iryna Gurevych. Adapterdrop: On the efficiency of adapters in transformers. *arXiv preprint arXiv:2010.11918*, 2020.
- Manuel Senge, Timour Igamberdiev, and Ivan Habernal. One size does not fit all: Investigating strategies for differentially-private learning across nlp tasks. *arXiv preprint arXiv:2112.08159*, 2021.
- Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. Membership inference attacks against machine learning models. In *Proceedings of the 38th IEEE Symposium on Security and Privacy, SP '17*, pages 3–18, Washington, DC, USA, 2017. IEEE Computer Society.
- Shuang Song, Kamalika Chaudhuri, and Anand D Sarwate. Stochastic gradient descent with differentially private updates. In *Proceedings of the 2013 IEEE Global Conference on Signal and Information Processing, GlobalSIP '13*, pages 245–248, Washington, DC, USA, 2013. IEEE Computer Society.

- Pranav Subramani, Nicholas Vadivelu, and Gautam Kamath. Enabling fast differentially private sgd via just-in-time compilation and vectorization. In *Advances in Neural Information Processing Systems 34*, NeurIPS '21. Curran Associates, Inc., 2021.
- Zhiliang Tian, Yingxiu Zhao, Ziyue Huang, Yu-Xiang Wang, Nevin Zhang, and He He. SeqPATE: Differentially private text generation via knowledge distillation, 2022. URL https://openreview.net/forum?id=5sP_PUUS78v.
- Florian Tramèr and Dan Boneh. Differentially private learning needs better features (or much more data). In *Proceedings of the 9th International Conference on Learning Representations*, ICLR '21, 2021.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30*, NIPS '17, pages 5998–6008. Curran Associates, Inc., 2017.
- Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R Bowman. Glue: A multi-task benchmark and analysis platform for natural language understanding. In *International Conference on Learning Representations*, 2018.
- Adina Williams, Nikita Nangia, and Samuel Bowman. A broad-coverage challenge corpus for sentence understanding through inference. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, NAACL-HLT '18, pages 1112–1122. Association for Computational Linguistics, 2018.
- Da Yu, Huishuai Zhang, Wei Chen, and Tie-Yan Liu. Do not let privacy overbill utility: Gradient embedding perturbation for private learning. In *Proceedings of the 9th International Conference on Learning Representations*, ICLR '21, 2021a.
- Da Yu, Huishuai Zhang, Wei Chen, Jian Yin, and Tie-Yan Liu. Large scale private learning via low-rank reparametrization. In *Proceedings of the 38th International Conference on Machine Learning*, ICML '21. JMLR, Inc., 2021b.
- Yingxue Zhou, Zhiwei Steven Wu, and Arindam Banerjee. Bypassing the ambient dimension: Private SGD with gradient subspace identification. In *Proceedings of the 9th International Conference on Learning Representations*, ICLR '21, 2021.

Table 8: Test accuracy for fine-tuning RoBERTa-Large with different privacy parameters. The number of training samples is denoted by n . The values of σ are noise multipliers. Numbers in the brackets are the changes compared to the results in Table 5 ($\varepsilon = 6.7$, $\delta = \Theta(1/n)$).

Taks	σ	$\delta = 1/n$	$\delta = 1/10n$	$\delta = 1/100n$	$\delta = 1/1000n$	Accuracy (in %)
MNLI	1.88	$\varepsilon = 1$	$\varepsilon = 1.35$	$\varepsilon = 1.49$	$\varepsilon = 1.61$	86.8 (-1.0%)
QQP	1.88	$\varepsilon = 1$	$\varepsilon = 1.40$	$\varepsilon = 1.54$	$\varepsilon = 1.67$	85.2 (-2.2%)
QNLI	3.01	$\varepsilon = 1$	$\varepsilon = 1.48$	$\varepsilon = 1.64$	$\varepsilon = 1.79$	88.0 (-2.8%)
SST-2	3.63	$\varepsilon = 1$	$\varepsilon = 1.47$	$\varepsilon = 1.64$	$\varepsilon = 1.80$	93.1 (-2.2%)
MNLI	0.91	$\varepsilon = 3$	$\varepsilon = 4.12$	$\varepsilon = 4.51$	$\varepsilon = 4.89$	87.4 (-0.4%)
QQP	0.93	$\varepsilon = 3$	$\varepsilon = 4.10$	$\varepsilon = 4.49$	$\varepsilon = 4.86$	86.8 (-0.6%)
QNLI	1.29	$\varepsilon = 3$	$\varepsilon = 4.45$	$\varepsilon = 4.90$	$\varepsilon = 5.33$	89.9 (-0.9%)
SST-2	1.52	$\varepsilon = 3$	$\varepsilon = 4.37$	$\varepsilon = 4.83$	$\varepsilon = 5.25$	94.1 (-1.2%)

A Experiments with Different Privacy Parameters

Now we test our framework under different privacy constraints. Specifically, we run LoRA on the language understanding tasks with various choices of privacy parameters ε and δ . We consider both RoBERTa-Base and RoBERTa-Large.

For the RoBERTa-Large model, we set $\varepsilon = 1$ and 3 with δ being the same as those in Section 4. We use the PRV accountant (Gopi et al., 2021). After getting the noise multipliers, we also reduce the value of δ and report the corresponding value of ε . The hyperparameters are the same as those in Section 4. We run experiments on all four tasks, i.e., MNLI ($n \sim 392k$), QQP ($n \sim 364k$), QNLI ($n \sim 104k$), and SST-2 ($n \sim 67k$). We report the results in Table 8. The performance of our framework is decent even with very tight privacy budgets. For instance, with $\varepsilon < 2$ and $\delta = 1/1000n$, the accuracy gap between the non-private baseline is only 3.8 for MNLI and 2.1 for SST-2.

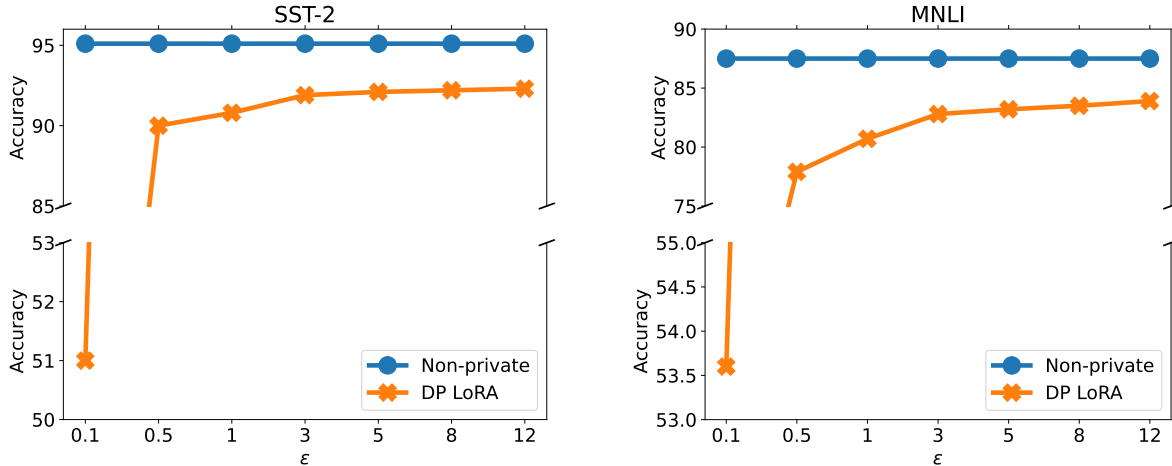


Figure 2: Test accuracy (in %) of fine-tuning the RoBERTa-Base model on MNLI and SST-2 with various choices of ε .

For the RoBERTa-Base model, we try various choices of ε . The values of ε are chosen from $[0.1, 0.5, 1, 3, 5, 8, 12]$. All other settings are the same as those in Section 4. We run experiments on the MNLI and SST-2 datasets. The results are presented in Figure 2. Our framework performs well for a wide range of ε . We note that our algorithm achieves meaningful accuracy even for very tight privacy parameters $\varepsilon = 0.5$ and 1. Such values of ε are rarely explored when training deep models with differential privacy.

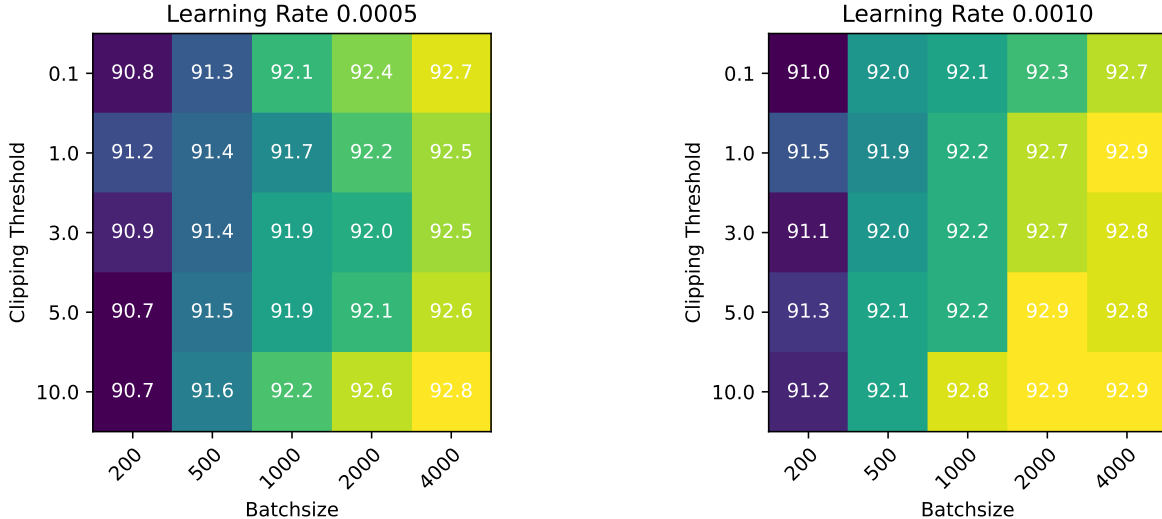


Figure 3: Test accuracy (in %) of fine-tuning RoBERTa-Base with differentially private LoRA on the SST-2 dataset. Our algorithm performs well on a wide range of hyperparameters.

Table 9: Accuracy for fine-tuning downstream tasks with RoBERTa-Base (in %). Experiments are run with full-precision. We also scale up the batch size according to the dataset size compared to SST-2. The privacy parameters are $\varepsilon = 6.7$, and $\delta = 1e-5$ for SST-2 and QNLI and $1e-6$ for MNLI and QQP.

Method		MNLI	SST-2	QQP	QNLI	Average Accuracy
Full	w/o DP	87.6	94.8	91.9	92.8	91.8
	DP	83.2	85.9	86.2	84.8	85.0
Adapter	DP	84.6	92.9	87.4	89.2	88.5
LoRA	DP	84.5	92.7	87.1	88.3	88.2

B On the Influence of Hyperparameters

Here we demonstrate that our algorithms perform well for a wide range of hyperparameters. We study two hyperparameters that are directly related to the variance of noise: clipping threshold and batchsize. The clipping threshold is chosen from $[0.1, 1.0, 3.0, 5.0, 10.0]$ and the batchsize is chosen from $[200, 500, 1000, 2000, 4000]$. We note that we keep the number of updates the same as that in Section 4 when the batchsize is changed. We fine-tune the RoBERTa-Base model with differentially private LoRA ($r = 16$) on the SST-2 dataset. The results are presented in Figure 3. DP LoRA performs well for all the hyperparameters considered. The gap between the best accuracy and the worst accuracy is only 2%.

C Fine-Tuning for Language Understanding Tasks with Large Batch Size and Full-Precision

Li et al. (2022) show the performance of fine-tuning the full model is sensitive to the choice of hyperparameters. They give a configuration which can significantly improve the performance of full fine-tuning. In this section, we re-evaluate the tasks in Table 4 and 5 under the configuration in Li et al. (2022).

The configuration in Li et al. (2022) has two differences compared to that in Section 4. The first difference is Li et al. (2022) run experiments with full-precision while the experiments in Section 4 use half-precision. Using half-precision is a common approach to speed up NLP experiments (Ott et al., 2018). However, half-precision may incur underflow issue which impacts the model performance (Micikevicius et al., 2017). The second difference is they use larger batch size for larger datasets. For example, the batch size for MNLI is

Table 10: Accuracy for fine-tuning downstream tasks with RoBERTa-Large (in %). Experiments are run with full-precision. We also scale up the batch size according to the dataset size compared to SST-2. The privacy parameters are $\varepsilon = 6.7$, and $\delta = 1\text{e-}5$ for SST-2 and QNLI and $\delta = 1\text{e-}6$ for MNLI and QQP.

Method		MNLI	SST-2	QQP	QNLI	Average Accuracy
Full	w/o DP	90.2	96.4	92.2	94.7	93.4
	DP	86.4	90.9	87.5	89.4	88.6
Adapter	DP	88.6	94.5	87.8	91.6	90.6
LoRA	DP	89.0	95.3	88.4	92.4	91.3

roughly six times larger than the batch size for SST-2 in Li et al. (2022). In Section 4, we use the same batch size for all datasets.

We follow the above setup and re-evaluate DP-LoRA and DP-Adapter. The results are in Table 9 and 10. The results of full fine-tuning with differential privacy are directly adopted from Li et al. (2022). The configuration in Li et al. (2022) further improves the strong results in Table 4 and 5. For example, we achieve 89.0% accuracy on the MNLI dataset, which is only 1.2% lower than the accuracy without DP constraint. Moreover, the benefit of the proposed framework over full fine-tuning is still clear. The average accuracy of the proposed algorithms is $\sim 3\%$ higher than that of full fine-tuning.